МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Языки программирования

Оконное приложение

Руководитель _____ А.В. Проскурин
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

Получение практических навыков разработки и отладки программ с графическим интерфейсом.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Изучите теоретические сведения к данной лабораторной работе.
2. Ознакомиться с постановкой задачи (вариант задания вычисляется как ваш_номер_в_списке_группы % количество_вариантов).
3. Составить программу, решающую поставленную задачу.
4. Отладить программу на подготовленных наборах тестовых данных.
5. Подготовить отчет по лабораторной работе. Отчет должен включать в себя:
   - постановку задачи;
   - текст программы с комментариями;
   - наборы тестовых исходных данных с соответствующими результатами работы программы;
   - краткие ответы на контрольные вопросы
6. Защитить лабораторную работу перед преподавателем

ПОСТАНОВКА ЗАДАЧИ

Создать оконное приложение, которое позволит выполнять следующие действия:

1. По выбору пользователя вводить список чисел с клавиатуры или из файла.
2. Выполнять действия согласно варианту задания.
3. Выводить результат работы и по выбору пользователя сохранять его в файл.

Вариант 5. В списке с числами X[N] найти произведение ненулевых элементов, а также количество элементов, значение которых превышает среднее арифметическое значение всех элементов списка.

ХОД РАБОТЫ

Файл **header**.py:

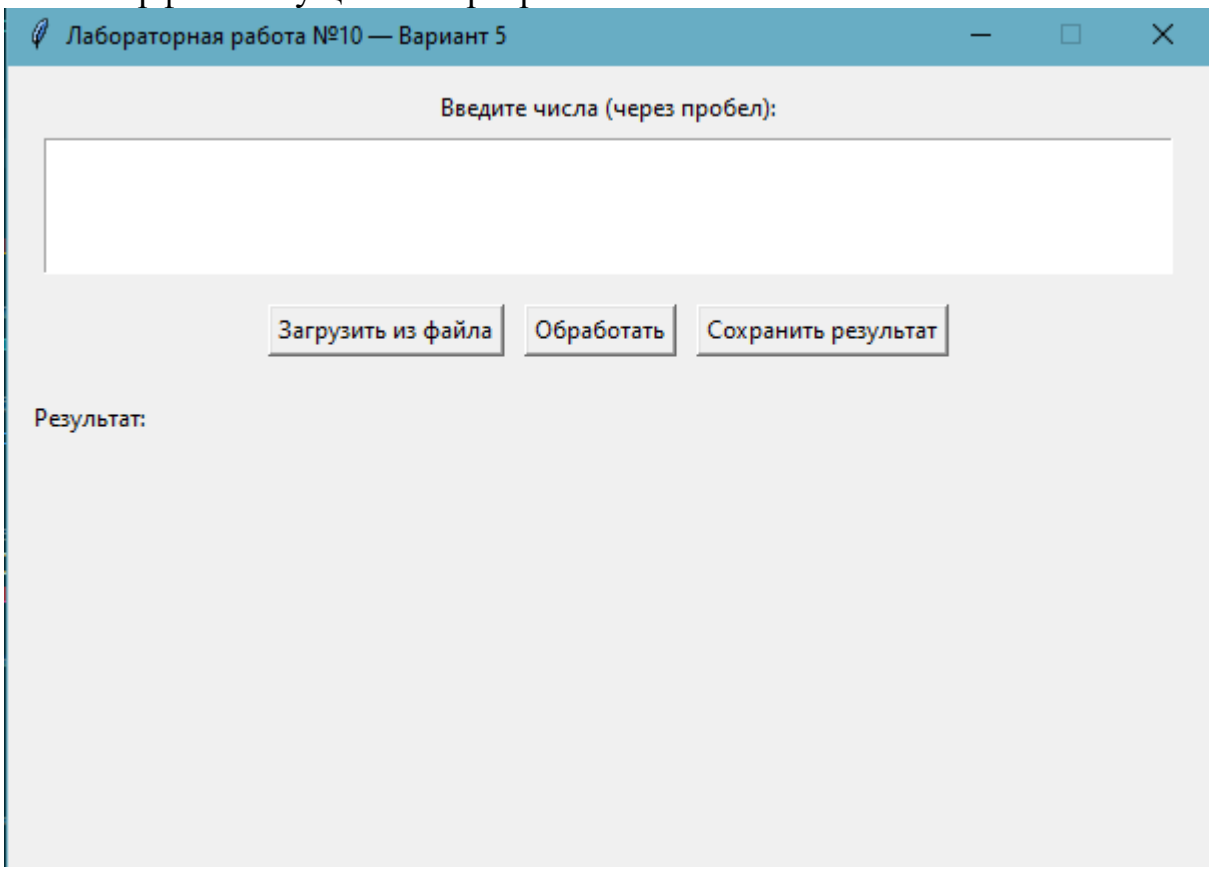
```python
def calculate_product_and_count_above_average(numbers):
    if not numbers:
        return 1, 0  # по умолчанию: произведение ненулевых — 1; кол-во элементов выше среднего — 0

    # Произведение ненулевых элементов
    product = 1
    for x in numbers:
        if x == 0:
            continue
        product *= x

    # Среднее арифметическое
    average = sum(numbers) / len(numbers)

    # Количество элементов, превышающих среднее
    count_above_avg = sum(1 for x in numbers if x > average)

    return product, count_above_avg
```

Файл **main.py**:

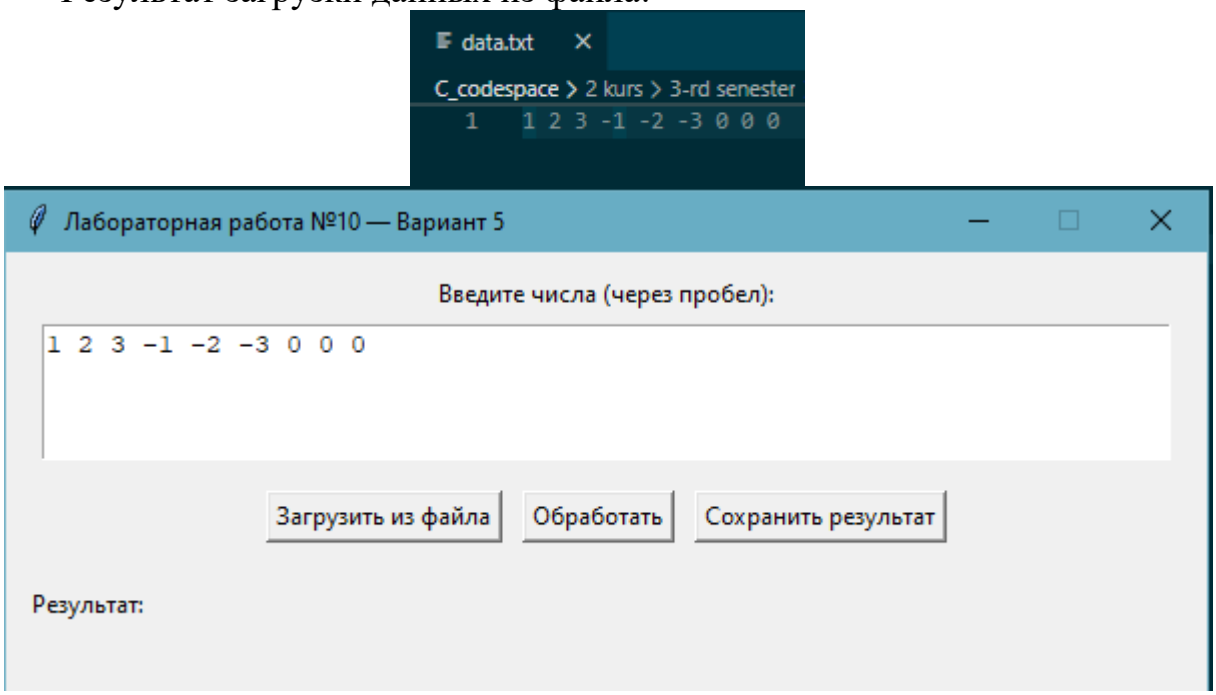
```python
import tkinter as tk
from tkinter import filedialog, messagebox
from header import calculate_product_and_count_above_average

def load_from_file():
    # Открываем окно выбора файла
    filepath = filedialog.askopenfilename(filetypes=[("Text files", "*.txt")])

    if not filepath:
        return

    with open(filepath, 'r', encoding='utf-8') as f:
        content = f.read().strip()

    # Пытаемся преобразовать содержимое в список целых чисел
    try:
        numbers = list(map(int, content.split()))
        # Очищаем текстовое поле и вставляем прочитанные числа через пробел
        input_text.delete(1.0, tk.END)
        input_text.insert(tk.END, ' '.join(map(str, numbers)))
    except Exception:
        messagebox.showerror("Ошибка", "Неверный формат файла")


# Функция обработки введённых данных
def process_data():
    raw = input_text.get(1.0, tk.END).strip()

    if not raw:
        result_label.config(text="Результат: введите данные")
        return

    numbers = list(map(int, raw.split()))
    product, count = calculate_product_and_count_above_average(numbers)

    result_label.config(text=f"Результат:\nПроизведение ненулевых: {product}\nКол-во выше среднего: {count}")

# Сохранение результата вычислений в файл
def save_result():
    raw = input_text.get(1.0, tk.END).strip()

    if not raw:
        messagebox.showwarning("Предупреждение", "Нет данных для сохранения")
        return

    numbers = list(map(int, raw.split()))
    product, count = calculate_product_and_count_above_average(numbers)

    # Открываем диалог сохранения файла
    filepath = filedialog.asksaveasfilename(
        defaultextension=".txt",
        filetypes=[("Text files", "*.txt")]
    )

    if not filepath:
        return

    with open(filepath, 'w', encoding='utf-8') as f:
        f.write(f"Произведение ненулевых элементов: {product}\n")
        f.write(f"Количество элементов выше среднего: {count}\n")
```

```python
            f.write(f"Количество элементов выше среднего: {count}\n")


# Создание главного окна приложения
root = tk.Tk()
root.title("Лабораторная работа №10 — Вариант 5")
root.geometry("600x400")
root.resizable(False, False)

# Метка над текстовым полем ввода
tk.Label(root, text="Введите числа (через пробел):").pack(pady=(10, 0))

# Текстовое поле для ввода данных
input_text = tk.Text(root, height=4, width=70)
input_text.pack(pady=5)  # Отступы сверху и снизу

# Контейнер для расположения кнопок
frame_buttons = tk.Frame(root)
frame_buttons.pack(pady=10)

# Размещение кнопок в контейнере
tk.Button(frame_buttons, text="Загрузить из файла", command=load_from_file).pack(side=tk.LEFT, padx=5)
tk.Button(frame_buttons, text="Обработать", command=process_data).pack(side=tk.LEFT, padx=5)
tk.Button(frame_buttons, text="Сохранить результат", command=save_result).pack(side=tk.LEFT, padx=5)

# Метка для вывода результата
result_label = tk.Label(
    root,
    text = "Результат:",
    justify = tk.LEFT,
    anchor = "w", # Якорь по левому краю
)
result_label.pack(pady = 10, padx = 10, anchor = "w")

# Запуск цикла обработки Tkinter
root.mainloop()
```

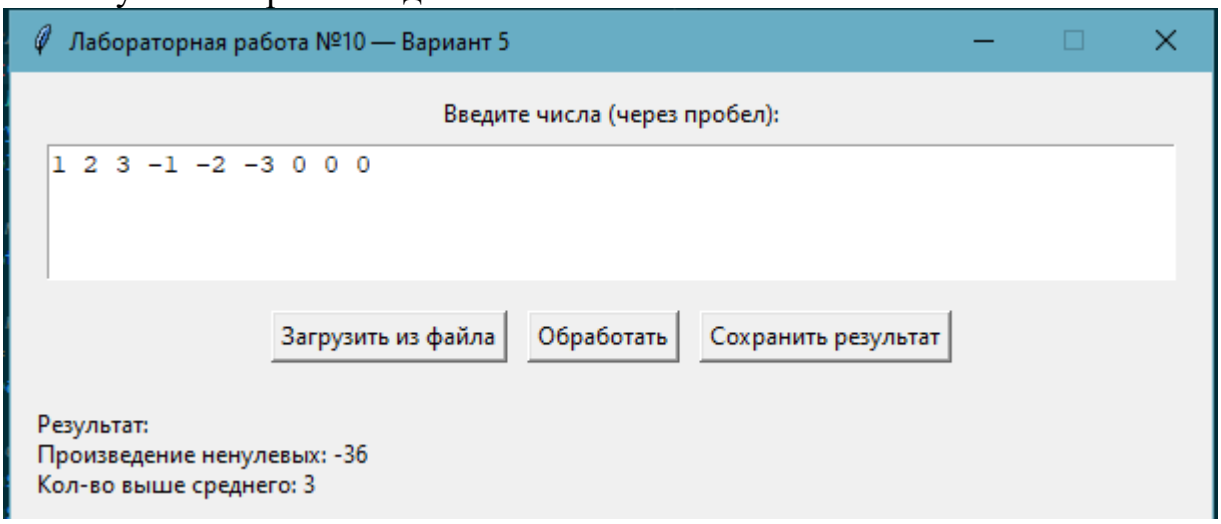
Интерфейс запущенной программы:



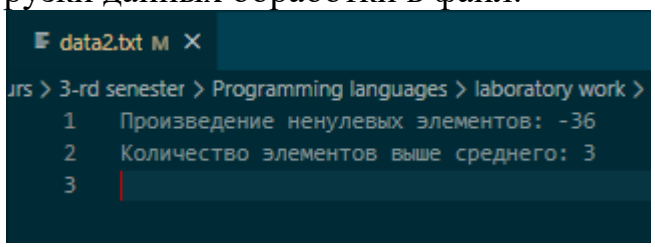
Результат загрузки данных из файла:



Результат обработки данных:



Результат выгрузки данных обработки в файл:



ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. **Как создать окно в Python с помощью Tkinter?**
   - Подключить библиотеку tk: **import tkinter as tk**
   - Создать окно с помощью класса TK: **root = tk.Tk()**
2. **Какой метод используется для отображения окна и взаимодействия с пользователем?**
   root.mainloop() — запускает цикл обработки событий, отображающий окно и ожидающей действий пользователя.
3. **Какие методы и атрибуты класса Tk позволяют установить размеры и начальную позицию окна?**
   - Размеры: **geometry(“*ширинахвысота* ”)**
   - Позиция: **root.geometry(“*ширинахвысота*+*X*+*Y*”)**
4. **Как получить текущие размеры и позицию окна?**
   Метод **winfo_geometry()**, который возвращает строку с данными окна.
5. **Как запретить изменение размера окна?**
   Метод **root.resizable(False, False)**
6. **Как установить заголовок окна?**
   Метод **title()**: **root.title(“Название окна”)**
7. **Какие виджеты доступны в Tkinter для создания графического интерфейса?**
   - **Label** — метка (текст).
   - **Button** — кнопка.
   - **Entry** — однострочное текстовое поле.
   - **Text** — многострочное текстовое поле.
   - **Frame** — контейнер для других виджетов.
   - **Checkbutton, Radiobutton** — флажки и переключатели.
   - **Listbox, Scrollbar** — списки и полосы прокрутки.
8. **Как позиционировать виджеты внутри окна?**
   - **pack()** — упаковка по сторонам.
   - **grid()** — сетка (строки и столбцы).
   - **place()** — абсолютное позиционирование по координатам.
9. **Как обрабатывать события, такие как нажатия кнопок или изменения текста в текстовых полях?**
   С помощью привязки функции-обработчика в параметрах элемента:
   btn = tk.Button(root, text="Нажми меня", *command=command_name*)
10. **Опишите процесс создания графического интерфейса с использованием Tkinter.**
    1. Импортировать библиотеку: **import tkinter as tk**
    2. Создать главное окно: **root = tk.Tk()**
    3. Настроить параметры окна: заголовок, размер, запрет изменений
    4. Создать виджеты
    5. Разместить виджеты в окне

6. Привязать функции-обработчики событий
7. Запустить цикл событий

11. **Какие ключевые объекты используются в работе с Tkinter?**
    - **Tk** — главное окно приложения
    - **Widget** — базовый класс для всех виджетов
    - **Frame** — контейнер для группировки виджетов
    - **StringVar**, **IntVar** — переменные для связи с виджетами
    - **Event** — объект события

ВЫВОДЫ

Я получил практические навыки разработки и отладки программ на языке Python, использующих графический интерфейс.